

AULA 2

Laboratório: Primeira API REST

Disciplina: CK0207 — Desenvolvimento de Software para Web

Professor: Dr. Fernando Antônio Mota Trinta

Universidade Federal do Ceará (UFC)

Projeto do curso: `nest-api-curso`

Autor: Samuel Sales Furtado

Objetivos de Aprendizagem

- Criar um Controller do zero, com rotas GET e POST
- Criar um Service para concentrar a regra de negócio
- Implementar endpoints REST seguindo boas práticas (verbos e status codes corretos)
- Testar cada endpoint criado, direto no navegador ou via terminal

Esta é a primeira aula de laboratório do curso. O projeto criado aqui — `nest-api-curso`, com o domínio **Users** — continua existindo e crescendo nas aulas seguintes; nenhuma aula posterior recria o projeto do zero.

1. Criação de Controllers

O Controller é a classe decorada com `@Controller('users')` — o texto entre parênteses define o prefixo da rota, de modo que tudo declarado dentro dela responde em `/users`. Cada método é decorado com o verbo HTTP correspondente: `@Get()` para buscar, `@Post()` para criar, e assim por diante.

```
@Controller('users')
export class UsersController {
  @Get()
  findAll() {}

  @Get('/:id')
  findOne(@Param('id') id: string) {}

  @Post()
  create(@Body() dto: any) {}
}
```

1.1 Decorators de parâmetro

Decorator	Papel
<code>@Param('id')</code>	Extrai um parâmetro da URL (ex: <code>/users/3</code>)
<code>@Body()</code>	Extrai o corpo da requisição (POST/PUT/PATCH)
<code>@Query('nome')</code>	Extrai a query string (<code>?nome=valor</code>)

Nesta aula, o `@Body()` ainda recebe o payload sem validação — apenas como um objeto qualquer. Validação de verdade, com DTOs, é assunto da Aula 3.

Um detalhe relevante para quando o projeto crescer: a ordem de declaração dos métodos no controller importa quando existem rotas parecidas, como `/users/me` e `/users/:id`. Rotas mais específicas precisam vir antes das rotas com parâmetro genérico, senão o Nest tentaria casar `"me"` como se fosse um `id`.

Vale reforçar o princípio central: **o controller não processa nada** — ele apenas recebe a requisição, chama o método certo do service, e devolve o que o service retornar. Um `if` de regra de negócio dentro do controller é sinal de que aquela lógica deveria estar no service.

2. Criação de Services

```

@Injectable()
export class UsersService {
  private users = [];
  private idCounter = 1;

  create(data) {
    const user = { id: this.idCounter++, ...data };
    this.users.push(user);
    return user;
  }

  findAll() {
    return this.users;
  }
}

```

O array em memória simula o "banco" por enquanto — sem a distração de configurar infraestrutura de banco de dados nesta fase do curso. Quando esse array for trocado por Prisma, na Aula 5/6, **o Controller não precisará mudar uma linha sequer** — apenas os métodos internos do Service passarão a conversar com o banco de verdade. Essa é a separação de responsabilidades definida na Aula 1 se pagando na prática.

O Controller recebe esse Service **injetado no construtor** — nunca com `new UsersService()`. Como o Nest mantém uma única instância do Service (o padrão singleton visto na Aula 1), o array `users` guarda os dados entre requisições enquanto o servidor estiver rodando; ao reiniciar o servidor, os dados somem, pois ainda não há persistência real.

3. Implementação de Endpoints REST

Os três endpoints desta aula:

```

GET /users      → lista todos os usuários
GET /users/:id  → retorna um usuário específico
POST /users     → cria um novo usuário

```

3.1 Boas práticas desde já

- **GET nunca deve alterar estado** — só lê dado, nunca escreve.
- O Nest devolve `201 Created` automaticamente em handlers `@Post()`; os demais verbos devolvem `200` por padrão.
- **Devolva o recurso criado/atualizado** no corpo da resposta.
- **Nomeie rotas no plural** (`/users`, não `/user`).

Os métodos do controller também podem ser `async` e retornar uma `Promise` — o Nest resolve isso automaticamente antes de montar a resposta. Nesta aula os métodos são síncronos porque o "banco" é um array em memória; a partir da Aula 5, esses mesmos métodos passam a ser `async`, sem alterar a estrutura do controller. O Nest também serializa automaticamente qualquer objeto/array devolvido pelo handler em JSON, com o `Content-Type` correto — não é necessário chamar `res.json()` manualmente, como em Express puro.

Laboratório — Passo a Passo ## Passo 1 — Criar o projeto do curso

```

nest new nest-api-curso
cd nest-api-curso
npm run start:dev

```

Testar: abrir `http://localhost:3000` — deve aparecer `Hello World!`.

Passo 2 — Gerar módulo, controller e service

```
nest g module users
nest g controller users
nest g service users
```

Passo 3 — Editar o `users.service.ts`

```
import { Injectable } from '@nestjs/common';

export interface User {
  id: number;
  name: string;
  email: string;
}

@Injectable()
export class UsersService {
  private users: User[] = [];
  private idCounter = 1;

  create(data: Omit<User, 'id'>): User {
    const user: User = { id: this.idCounter++, ...data };
    this.users.push(user);
    return user;
  }

  findAll(): User[] {
    return this.users;
  }

  findOne(id: number): User | undefined {
    return this.users.find((user) => user.id === id);
  }
}
```

Passo 4 — Editar o `users.controller.ts`

```
import { Controller, Get, Post, Body, Param } from '@nestjs/common';
import { UsersService } from '../users.service';

@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Post()
  create(@Body() createUserDto: { name: string; email: string }) {
    return this.usersService.create(createUserDto);
  }

  @Get()
  findAll() {
    return this.usersService.findAll();
  }

  @Get('/:id')
  findOne(@Param('id') id: string) {
    return this.usersService.findOne(+id);
  }
}
```

Passo 5 — Testar os três endpoints

```
curl -X POST http://localhost:3000/users \
-H "Content-Type: application/json" \
-d '{"name":"Ana Souza","email":"ana@example.com"}'
```

Esperado: {"id":1,"name":"Ana Souza","email":"ana@example.com"} com status 201 .

```
curl http://localhost:3000/users
curl http://localhost:3000/users/1
```

O que ainda falta (gancho para as próximas aulas)

- O `POST /users` aceita qualquer coisa no corpo, sem validar nada — corrigido na **Aula 3**, com DTOs e `ValidationPipe`.
- Buscar um `id` inexistente devolve vazio, sem erro HTTP apropriado — corrigido na **Aula 4**, com tratamento de erros e CRUD completo.
- Os dados vivem apenas na memória — corrigido na **Aula 5/6**, com Prisma.

Verificação de Conformidade

- ✔ `@Controller()`, `@Get()`, `@Post()`, `@Param()`, `@Body()` — conferido em docs.nestjs.com/controllers.
- ✔ Status code 201 padrão em `@Post()`, 200 nos demais — conferido em docs.nestjs.com/controllers.
- ✔ `@Injectable()` e injeção via construtor — conferido em docs.nestjs.com/providers.
- ⚠ `@Body()` tipado como objeto literal simples em vez de DTO — decisão pedagógica intencional desta aula, corrigida na Aula 3.